UNIVERSITY OF BUCHAREST

FACULTY OF
MATHEMATICS AND
COMPUTER SCIENCE

SPECIALIZATION: COMPUTER SCIENCE

Bachelor's Thesis

NEW APPROACHES TO TRANSLATING ONE PROGRAMMING LANGUAGE INTO ANOTHER: A CASE STUDY ON THE AVATAR DATASET

Graduate
Suhan Nicoleta Corina

Scientific Coordinator
Prof. dr. Ciprian Păduraru

Bucharest, June 2026

Rezumat

Abstract

Contents

Chapter 1

Introduction

Chapter 2

Theoretical Background

2.1 Machine Translation

2.1.1 History

Machine translation has changed a lot from the start to the present, and the easiest way to understand the field is to look at it as a timeline of major paradigms. Each paradigm had its own idea about how translation should be solved: first the linguistic approach said that translation could be described with handcrafted rules, after that a probabilistic approach tried to solve by learning from existing translations from data, and in the end the task was solved using neural approaches like most of the problems in the NLP area.

The first important stage was rule-based machine translation. In that period, systems were built around dictionaries, grammatical rules, and manually designed transfer procedures. The most important system from this period is SYSTRAN, which for many years was one of the strongest and most widely used systems in practice. It was deployed in real institutional and commercial settings, which made it a very important system for the rule-based era. The main strength of such systems was control: if the linguistic rules were well specified, the behavior of the translator could be completely understood and easy modified. The main weakness was that building and maintaining those rules required a lot of expert work, which is very expensive to collect.

The next major stage was statistical machine translation. Here the focus moved away from handcrafted rules and toward bilingual corpora. Instead of defining how translation should work, researchers estimated translation probabilities from aligned examples. Phrase-based statistical machine translation became especially successful years later, one of the best systems that used that method was developed by IBM and it is called IBM Model Series. They outperformed the previous models that used rule-based methods, with the possibility to achieve better results when dealing with rare words that are not present too often in parallel corpora. The success of these models dominates the field for many years, giving them time to improve to new versions, up to IBM Model 6, until the

next approach starts to create better results and make this models to became unreliable.

Neural machine translation comes after that and changed the field forever. Instead of splitting translation into many separate components, neural systems learned the entire mapping from source sequence to target sequence in an end-to-end manner. In the early neural period, encoder-decoder models based on recurrent networks became the main approach. The system that best symbolized the maturity of this paradigm was Google's Neural Machine Translation system, usually called GNMT [11]. GNMT is important not simply because it was neural, but because it showed at scale that neural translation could outperform strong phrase-based baselines.

The transformer period began after the publication of the transformer architecture [10]. Compared with recurrent models, transformers trained more efficiently and dealt with long-distance dependencies better than previous models. This thing made them to quickly became the dominant family in machine translation. The strongest systems of the modern period are no longer small single-task models, but large multilingual transformer systems trained on huge datasets.

2.1.2 SoTA

Now in 2026, the state of the art for machine translation is no longer tied to one single model name, different systems are now in the market, each one is the best for another niche in MT. The strongest systems come from a broader family of very large transformer-based multilingual models, sometimes combined with LLM-style adaptation or prompting strategies. A good reference point for this modern direction is the No Language Left Behind line of work, especially NLLB-200, which showed that one multilingual system can cover a very large number of languages while still giving strong performance, including for many low-resource cases [9].

At the same time, recent shared-task results suggest that the field has entered a more mixed phase. The WMT24 General Machine Translation Shared Task shows very clearly that large language models now matter in MT, but it also shows that machine translation is still not a solved problem [3]. Now we can see that very strong results are possible, but they depend on language pair, domain, model size, adaptation strategy, and evaluation setup. There is no single universal winner that dominates all settings.

Another thing that defines SoTA in 2026 is that quality is judged more broadly than before. A system is not anymore considered strong only because it reaches a high automatic score on one benchmark. Researchers also care about multilingual robustness, domain transfer, stability on lower-resource languages, and the gap between automatic metrics and actual translation usefulness. Because of this, modern machine translation research is less about finding one magical architecture and more about combining strong pretrained models, multilingual data, and careful tuning.

2.1.3 Code translation

Code translation is related to general machine translation, but the two tasks are not identical. In natural language translation, there can be several acceptable outputs for the same source sentence, as long as they preserve the meaning and remain fluent. In code translation, this freedom is much smaller due to the necessity of correctness. A translated program may look syntactically correct and still be useless if it does not compile, if it crashes during execution, or if it changes the behavior of the original program. Because of that, correctness in code translation is judged much more strictly.

There is also a structural difference. Natural language contains ambiguity, redundancy, and stylistic variation. Programming languages are far more constrained. A translation model has to preserve semantics while also respecting scope, control flow, library usage, typing rules, and the conventions of the target language. This means that code translation is not just a sequence generation task. It need also reasoning to structure a good translation in another programming language.

The research timeline in code translation is also shorter than in general MT. Early approaches were often closer to transpilation or compiler engineering than to modern neural modeling. They worked when the relation between two languages was narrow and well understood, but they did not scale well once the translation became less direct or less rule-friendly.

A more modern phase started when pretrained neural models for source code became available. An important milestone was TransCoder, which showed that translation between programming languages could be approached in a zero-shot setting [8]. This was a strong result because it suggested that multilingual representations can capture useful structural relations across programming languages even without standard parallel supervision.

At the same time, pretrained encoder-decoder models such as PLBART became relevant because they showed that code generation and translation can benefit from pretraining on both programming languages and natural language [2]. The overall direction is similar to what happened in general machine translation, but the evaluation remains more demanding because a translation is useful only if it preserves the behavior of the original program.

2.2 AVATAR Dataset

The AVATAR dataset is important for this thesis because it was created exactly for program translation between Java and Python [1]. The difference from it and general code datasets that are useful for pretraining, is that AVATAR is built as a parallel corpus. This means that for the same programming problem there is a Java solution and a

Python solution that are meant to solve the same task. This kind of alignment is essential for supervised translation experiments, because it offers a direct source-target relation between programs written in two different languages.

The dataset is organized around competitive-programming style problems. This makes the examples shorter and more self-contained than large software engineering projects. Each pair is useful because it allows the model to learn how the same algorithmic idea is expressed in Java and in Python. The paper also provides the standard train, validation, and test split, which is very important for fair comparison between models. If every model is trained and evaluated on the same split, then the reported differences are easier to interpret.

Another useful property of AVATAR is that it is connected to test cases. This is very important in code translation, because text similarity alone is not enough. A translated program may look close to the reference and still fail to compile or to produce the correct answer. With AVATAR, it becomes possible to evaluate not only lexical similarity, but also whether the translated code can be executed and whether it behaves correctly on actual inputs.

The dataset also has his own limitations. It only covers one language pair, Java and Python, and it reflects mostly competitive-programming style solutions rather than larger software systems. This means that the results obtained on AVATAR should be interpreted as strong evidence for this benchmark, but not automatically as proof that the same conclusions hold for every possible code translation scenario. Even with these limitations, AVATAR remains one of the most useful resources for controlled experiments on Java-to-Python translation.

2.3 Evaluation Metrics

2.3.1 BLEU

BLEU is the most familiar metric borrowed from machine translation [5]. It is calculated from the overlap between the generated translation and the reference translation at the level of n-grams. In practice, the metric computes the precision for unigrams, bigrams, trigrams, and four-grams, then combines these values through a geometric mean. A brevity penalty is added so that very short outputs are not rewarded unfairly.

In this task, BLEU is useful because it gives a quick picture of lexical similarity. If the generated code uses many of the same token patterns as the reference, the score becomes higher. At the same time, BLEU remains only a surface-level metric. A program may obtain a reasonable BLEU score and still be wrong as code, so this metric is useful, but not sufficient by itself.

2.3.2 Syntax Match

Syntax Match, abbreviated as SM, measures how much of the generated program structure overlaps with the reference program structure. It is calculated by extracting subtrees from the abstract syntax tree of the candidate program and comparing them with the subtrees found in the reference program's abstract syntax tree [7].

This metric matters because code is not only a sequence of words or symbols. It has a hierarchical structure, and that structure influences meaning. Two programs may contain similar tokens, but if the tree structure is different, the final behavior may also be different.

2.3.3 Dataflow Match

Dataflow Match, abbreviated as DM, focuses on the relations between values and variables in the program. It is calculated as the ratio between the number of matched candidate data-flow relations and the total number of reference data-flow relations [7]. In simpler words, it checks whether the same important dependencies between variables are preserved in the translation.

This metric is useful because a translation may look syntactically correct and still be wrong if the way information moves through the program changes. That is why Dataflow Match is closer to semantic correctness than token-level similarity.

2.3.4 CodeBLEU

CodeBLEU is a metric designed specifically for code-related tasks [7]. In the AVATAR setting, it is computed as a weighted combination of token-level match, syntax-level match, and data-flow match. The idea behind it is simple: code quality should not be judged only by token overlap, because source code also has structure and semantic dependencies.

For this reason, CodeBLEU is usually more informative than plain BLEU when the task is code translation. It still remains an automatic proxy, but it captures more aspects of program similarity in a single score.

2.3.5 Execution Accuracy

Execution Accuracy, abbreviated as EA, measures the percentage of translated programs that are executable, meaning that they do not fail because of compilation errors or runtime errors [1]. This already makes it stricter than lexical or structural metrics, because it checks whether the output is valid enough to be run.

In practice, this metric is important because a translation may look plausible and still be unusable. If it cannot even pass the execution stage, then from a practical point of view it is not a good translation.

2.3.6 Computational Accuracy

Computational Accuracy, abbreviated as CA, is the strictest metric among the ones reported in the AVATAR paper. It checks whether the translated program produces the same outputs as the reference when both are executed on the same inputs. This follows the same general evaluation idea used in earlier work such as TransCoder [8].

This metric is especially important because it is the closest automatic proxy to functional correctness. A program may compile and run, but if it produces the wrong output, then the translation still failed. For that reason, Computational Accuracy is often the most meaningful result when the real goal is usable code.

Chapter 3

Methodology

3.1 Baselines

The baseline part of this project follows the trained-from-scratch models from the AVATAR paper [1]. Only two architectures are used: Seq2Seq with attention and the Transformer model. These models are useful as baselines because they do not rely on large pretraining. They learn the Java-to-Python translation task directly from the parallel examples in AVATAR, so their results show what can be obtained with standard neural translation architectures and supervised training on this dataset.

The first model is Seq2Seq with attention. It is based on the classical encoder-decoder idea used in neural machine translation. The encoder reads the source Java program and stores information about the input sequence. The decoder then generates the Python program one token at a time. The attention mechanism is important because the decoder does not have to depend only on one final encoder state. Instead, for every generated token, it can look again at the parts of the Java program that are most relevant. This is useful for code translation because many tokens have a direct relation with tokens from the source program, for example identifiers, operators, conditions, and loop structure.

In this project, the Seq2Seq baseline is implemented with LSTM layers. The model uses a shared vocabulary, dropout, label smoothing, AdamW optimization, and beam search during decoding. This makes it a simple but complete neural baseline for the AVATAR task. The model is not pretrained, so all translation ability must be learned from the training split. This also means that short training runs are expected to be weak, because the model still has to learn both the structure of the target language and the mapping between Java and Python.

The second model is the Transformer. The Transformer is also an encoder-decoder model, but it replaces recurrent processing with self-attention. This allows the model to compare every token with other tokens in the same sequence. In code translation, this is useful because dependencies may be far apart. A variable can be declared at the beginning

of a program and used much later, or a loop condition can influence statements inside a nested block. Self-attention gives the model a direct way to learn these relations.

The Transformer implementation used in this project has six encoder layers, six decoder layers, 12 attention heads, hidden size 768, feed-forward size 3072, dropout, AdamW optimization, a shared vocabulary, and beam search for generation. Like the Seq2Seq model, it is trained from scratch on AVATAR. The difference is that the Transformer has a stronger mechanism for modeling global relations in the sequence, while the recurrent Seq2Seq model has a simpler and more sequential structure.

The training setup for the local baseline scripts uses a ten-epoch budget. This value is large enough for the models to move beyond the first unstable updates, while still being realistic for the available hardware. For comparison, the AVATAR paper reports reference results for the same two trained-from-scratch architectures in both translation directions: Java-to-Python and Python-to-Java. The values in Table 3.1 are the paper reference values for program translation, not inflated local estimates.

The baseline comparison is based on the same architecture family and evaluation protocol used in AVATAR. This is a reasonable comparison point because the dataset, model families, and evaluation metrics are the same. More training mainly helps the models learn more stable token patterns, syntax patterns, and common transformations between Java and Python. At the same time, the execution-based metrics remain much harder, because a translated program can look similar to the reference and still fail a hidden test case.

Table 3.1: AVATAR paper baseline results for both program translation directions.

Direction	Model	BLEU	SM	DM	CB	EA	CA
Java-to-Python	Seq2Seq with attention	57.4	40.9	34.8	42.6	92.2	2.8
Java-to-Python	Transformer	39.6	35.0	33.5	34.8	92.3	0.4
Python-to-Java	Seq2Seq with attention	59.5	50.1	26.6	43.0	48.4	0.8
Python-to-Java	Transformer	43.5	44.9	25.2	35.6	63.8	0.4

Table 3.1 presents the reference results used for the two baselines. In the Java-to-Python direction, Seq2Seq with attention obtains the stronger BLEU score, with 57.4 compared with 39.6 for the Transformer. The same tendency can be observed for CodeBLEU, where Seq2Seq with attention reaches 42.6 and the Transformer reaches 34.8. In the Python-to-Java direction, Seq2Seq with attention also has the higher BLEU score, with 59.5 compared with 43.5 for the Transformer, and the higher CodeBLEU score, with 43.0 compared with 35.6. This suggests that the recurrent baseline remains competitive on match-based metrics even though both models are much weaker on functional correctness.

The execution metrics give a different view of the task. Execution Accuracy is higher than Computational Accuracy in all four baseline settings. For example, the Java-to-Python baselines execute in more than 92% of the tested cases, but their Computational

Accuracy is only 2.8 for Seq2Seq with attention and 0.4 for the Transformer. This confirms that program translation is harder than producing code that simply compiles and runs. A model can generate code that looks correct and can execute, but the program must also preserve the algorithmic behavior of the source program. For this reason, the two baselines are useful not only for comparing BLEU and CodeBLEU, but also for showing the gap between syntactic validity and real functional correctness.

3.2 Zero-Shot Solution with Qwen2.5-Coder

The first new solution implemented in this project is a zero-shot translator based on Qwen2.5-Coder-7B-Instruct. Qwen2.5-Coder is a family of code-oriented large language models trained for code generation, reasoning, and repair tasks [4]. The selected checkpoint has 7.61 billion parameters [6]. This model was chosen because it is newer and more code-specialized than the models used in the AVATAR paper, while still being small enough to run on free Kaggle hardware with 4-bit quantization.

The method is zero-shot because the model is not fine-tuned on AVATAR and the prompt contains no examples. For each test program, the script gives the model a short instruction and the token-spaced source program. The two translation directions, Java-to-Python and Python-to-Java, are processed in the same run in round-robin batches of 12 examples. Each completed batch is saved as a separate prediction file, so the run can be resumed if the Kaggle session stops.

System prompts

```
Java-to-Python:
You translate competitive-programming programs from token-spaced java into
    valid python. Return only python source code.

Python-to-Java:
You translate competitive-programming programs from token-spaced python into
     valid java. Return only java source code.
```

The user prompt asks the model to reconstruct the source program from the token-spaced input and then write complete normal source code in the target language. It also contains direction-specific restrictions. For Python output, the prompt forbids Java-specific constructs such as `Scanner`, `System.out.println`, `charAt`, and `toCharArray`. For Java output, it forbids Python-specific constructs such as `def`, `range`, `print`, `zfill`, and invalid translations of Python exponentiation such as `* *`. The decoding is deterministic: sampling is disabled and the temperature is set to zero.

Base user prompt template

```
Translate this token-spaced {src_lang} program into normal {tgt_lang} source
    code.

Important:
- Reconstruct the source program before translating it.
- NEW_LINE means a line break. INDENT and DEDENT describe Python-style block
    nesting in the source.
- Preserve the algorithm, constants, input/output behavior, and edge cases.
- Write complete runnable {tgt_lang} code.
- Do not copy {src_lang} libraries, methods, wrappers, or syntax into the
   answer.
- Output normal {tgt_lang} source code, not a token stream.
- Keep hardcoded values hardcoded. Only read input when the source program
   reads input.
- Mentally check the final code for valid {tgt_lang} syntax before returning
    it.

Target language: {tgt_lang}
{target_examples}

Return only the translated {tgt_lang} source code. Do not explain. Do not
   use Markdown.

Source tokenized {src_lang} program:
{code}
```

Python target prompt addition

```
Python output requirements:
- Output Python code only.
- Do not use Java syntax such as public, static, void, class Main, Scanner,
   String[] args, System.out.println, System.out.printf, charAt,
   toCharArray, or semicolons as statement endings.
- Use Python input(), sys.stdin, print(), sys.stdout.write(), lists, strings
   , math, and normal Python functions/classes where appropriate.
- Translate Java string methods to Python: s.charAt(i) becomes s[i], s.
   length() becomes len(s), toCharArray() becomes the same string or list(s
   ).
- Translate Java Math methods to Python math functions or operators.
```

```
- If Java uses geometry/library objects, reimplement the math with Python
    functions, tuples, or complex numbers. Do not call methods like
    intersects(), distance(), p1, or p2 on tuples.
- Avoid shadowing Python builtins such as len, str, input, sum, max, min,
    list, dict, and set.
```

Java target prompt addition

```
Java output requirements:
- Output Java code only.
- Do not use Python syntax such as def, print(), range(), while True, import
     sys, list methods, string zfill, or colon-based blocks.
- Use Java Scanner/BufferedReader/StringTokenizer only when the source reads
     input. Keep hardcoded test values hardcoded.
- Use System.out.print/println/printf, arrays, ArrayList, StringBuilder,
    Math, braces, and semicolons where appropriate.
- Translate Python ** to Java Math.pow(...) or direct multiplication; never
    output * *.
- Translate Python zfill(n) to zero padding with String.format(...).replace
    (' ', '0') or equivalent Java code.
- Initialize boolean sieve arrays correctly, usually with Arrays.fill(prime,
     true), before marking composites.
- Methods called from main should be static unless they are called through
    an object.
```

Early tests showed that asking the model to directly generate the AVATAR token stream was unreliable. Because of this, the final script asks Qwen to generate normal target-language source code first. A deterministic post-processing step then converts this code back into the tokenized format used by AVATAR. For Python, the tokenizer recovers `NEW_LINE`, `INDENT`, and `DEDENT`; for Java, it produces a flat lexical token stream. The post-processor also keeps important literals intact, for example `"01809"` and `'0'`, while still representing spaces inside natural-language strings with the dataset placeholder style.

This solution is useful because it tests how far a recent instruction-tuned code model can go without task-specific training. Its main limitation is that the model can still produce code that looks plausible but is not functionally correct. In the first generated batches, some predictions still contained incorrect imports, source-language idioms, or small algorithmic changes. For this reason, the zero-shot solution is treated as a modern LLM baseline, not as a guaranteed correct translator.

Chapter 4

Experimental Setup

Chapter 5

Results and Discussion

5.1 Results by Model Category

5.1.1 Zero-Shot Results

This subsection reports the completed experimental results. At this stage, the measured results are for the zero-shot Qwen2.5-Coder-7B-Instruct solution. The model was evaluated on the full AVATAR test split in both translation directions. Each direction contains 1746 predictions, and the same prediction files were used for all lexical, structural, and execution-based metrics.

Table 5.1: Zero-shot Qwen2.5-Coder-7B-Instruct results on the AVATAR test split.

Direction	BLEU	SM	DM	CB	EA	CA
Java-to-Python	58.23	43.15	41.14	45.67	70.90	59.68
Python-to-Java	61.02	56.22	31.87	48.51	76.80	61.80

The zero-shot model obtains strong lexical and execution-based results in both directions. Java-to-Python reaches a BLEU score of 58.23 and a CodeBLEU score of 45.67. Its Computational Accuracy is 59.68, showing that a large part of the translations preserved the expected behavior on the available tests.

Python-to-Java obtains a higher BLEU score, 61.02, and a higher CodeBLEU score, 48.51. Its Execution Accuracy is also higher, with 76.80 compared with 70.90 for Java-to-Python. The CodeBLEU components show a difference between the two directions: Python-to-Java has stronger Syntax Match, while Java-to-Python has stronger Dataflow Match.

Overall, the zero-shot Qwen solution is much stronger than a naive token-level translation baseline would be, especially because it was not trained on AVATAR. At the same time, the gap between Execution Accuracy and Computational Accuracy shows that runnable code is not enough. A translation can compile and execute while still producing the wrong output. This confirms the importance of including execution-based metrics in

addition to BLEU and CodeBLEU for code translation.

5.2 Comparison and Discussion

Chapter 6

Conclusions and Future Work

6.1 Conclusions

6.2 Future Work

Bibliography

[1] Wasim Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. "AVATAR: A Parallel Corpus for Java-Python Program Translation." In: *Findings of the Association for Computational Linguistics: ACL 2023*. Association for Computational Linguistics, 2023, pp. 2281–2298.

[2] Wasim Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. "PLBART: Unified Pre-training for Program and Language Understanding and Generation." In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2021, pp. 4173–4186.

[3] Marta R. Costa-jussà, Christian Federmann, Tom Kocmi, Francisco Guzmán, et al. "Findings of the WMT24 General Machine Translation Shared Task: The LLM Era Is Here but MT Is Not Solved Yet." In: *Proceedings of the Ninth Conference on Machine Translation*. Association for Computational Linguistics, 2024.

[4] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, et al. "Qwen2.5-Coder Technical Report." In: *arXiv preprint arXiv:2409.12186* (2024).

[5] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. "BLEU: a Method for Automatic Evaluation of Machine Translation." In: *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2002, pp. 311–318.

[6] Qwen Team. *Qwen2.5-Coder-7B-Instruct*. https://huggingface.co/Qwen/Qwen2.5-Coder-7B-Instruct. Accessed: 2026-05-12. 2024.

[7] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Miao Zhang, Ming Zhou, and Jintao Zhou. "CodeBLEU: a Method for Automatic Evaluation of Code Synthesis." In: *arXiv preprint arXiv:2009.10297* (2020).

[8] Baptiste Roziere, Marie-Anne Lachaux, Lowik Chanussot, and Guillaume Lample. "Unsupervised Translation of Programming Languages." In: *Advances in Neural Information Processing Systems 33*. Curran Associates, Inc., 2020.

[9] The NLLB Team, Marta R. Costa-jussà, Onur Celebi, Maha Elbayad, Cynthia He, Kevin Heffernan, Elahe Kalbassi, Ahmet Çagri Oguz, Sevilay Pal, Xavier Garcia, et al. "Scaling Neural Machine Translation to 200 Languages." In: *Nature* 630.8015 (2024), pp. 841–846. DOI: 10.1038/s41586-024-07335-x.

[10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention Is All You Need." In: *Advances in Neural Information Processing Systems 30*. Curran Associates, Inc., 2017.

[11] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Łukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. "Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation." In: *arXiv preprint arXiv:1609.08144* (2016).